OOPs (Object Oriented Programming)

Introduction to Object Oriented Programming Principles (OOPs)

->In real Time, To develop any project / Application, we Use a Language and it can Satisfy to either Procedure Oriented Features (functional Programming) OR Object Oriented Features.

->In Other Words, we have Two Types of Programming languages. They are

1. Procedure Orineted(Functional) Programming Languges.

Examples:

Oracle7.3, Pascal, Cobol, c, PYTHON.

2. Object Urineted Programming Langugaes.

Examples:

c++, java, c.net, smalltalk, ruby, PYTHON, from Oraclel onwards

->Hence PYTHON Programming Lang Belongs to Both Functionele Oriented Programming lang.

->Even Python Programming Lang Belongs to both Functional and Object Oriented Program language, Internally Every Th

Advantages of Object Oriented Principles

1. Allow us to store large volume of data.
2. can get security ,bcoz internally data stored in the form of cipher-text format.
3. we can get effective communication , bcoz data transfered all at once.
4. provides minimal memory space ,bcoz oop is re-usable mechanism.
5. can get effective memory management, bcoz data always available in the form of objects.

Object Oriented Programming Principles

1. classes
2. Objects
3. Data Encapsulations
4. Data Abstraction
5. Inheritance
6. Polymorphism
7. Message Passing

ing treated as object.

OBJECT ORIENTED PROGRAMMING LANGUAGE (OOPs)

The concept of object allow us to store large amount of data.

Provides minimal memory space for application dev bcoz oops provides re-usable machanisms.

all data transfer at once.

PRINCIPLES OF OOPS

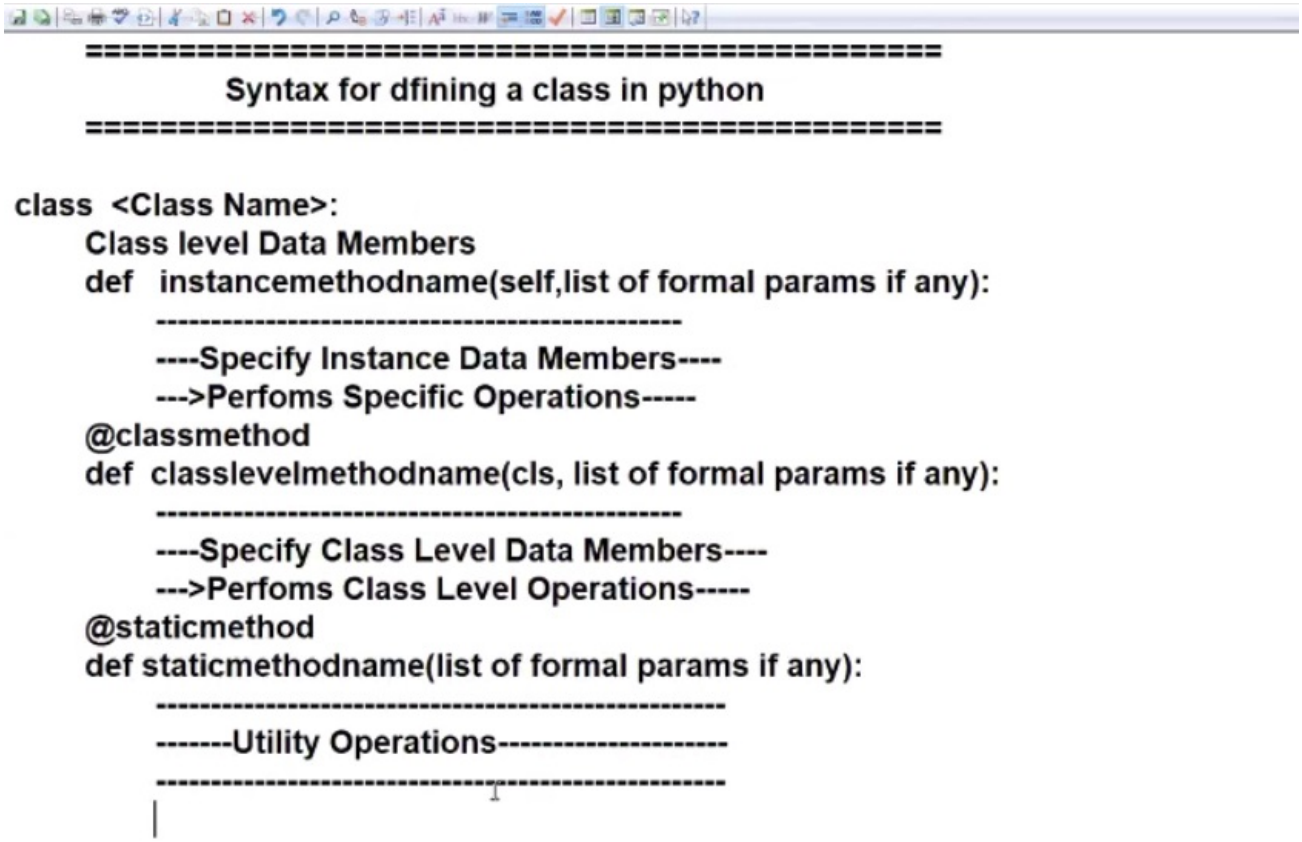
1.CLASSES:

The purpose of classes is that to dev programmer defined data types and to dev real time app by using keyword called 'class' hence every python program in oops must start with classes concept.

definition: class is a collection of data members(props) and methods(funs).

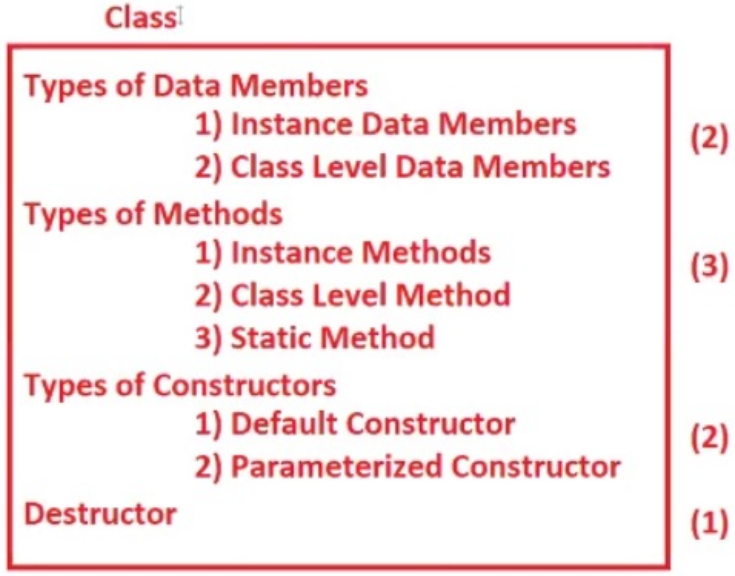
In [4]:
```
from IPython.display import Image
Image(r'C:\Users\shaba\Pictures\Screenshots\Screenshot (184).png')
```

Out[4]:



In [6]:
```
from IPython.display import Image
Image(r'C:\Users\shaba\Pictures\Screenshots\Screenshot (182).png')
```

Out[6]:



types of data members in class

i)instance data members

idm are those whose memory space created everytime during obj creation

used for storing specifc values inside of objests.can add inside objects in 3 ways..

a)by using an obj

b)by using instance methods

c)by using constructor

inside data members must be accesses w.r.t objname or self.

ii)class level data members

cldm are those whose memory space created one time irrespective of number of objs are created.

used for storing common value for all the objs of same class.can add members in two ways..

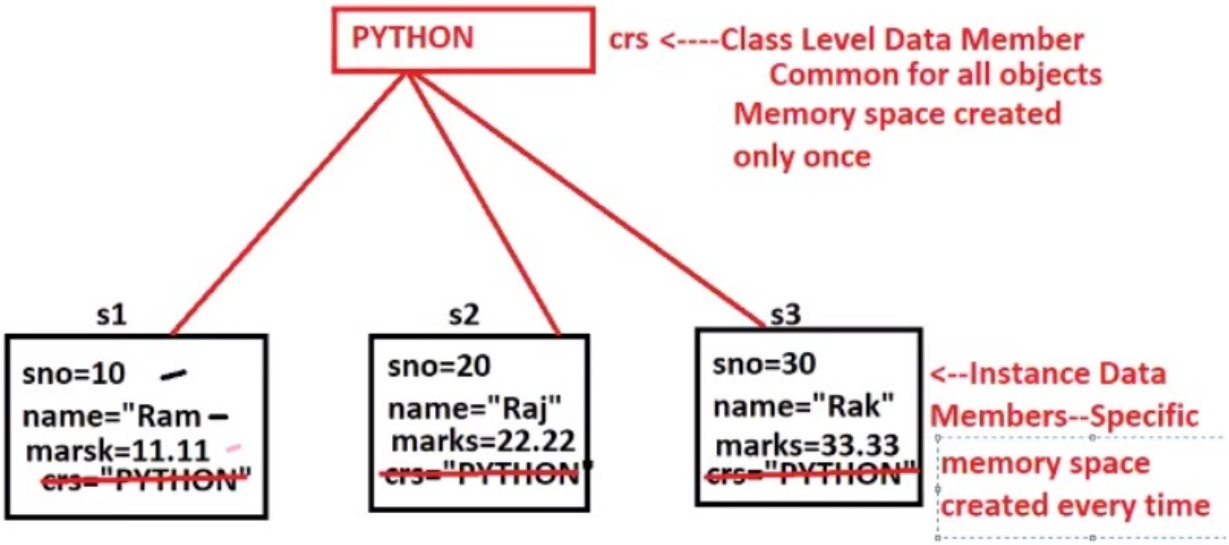
a)inside the class name

b)inside class level method

inside data members must be accesses w.r.t obj name or cls or object name or self.

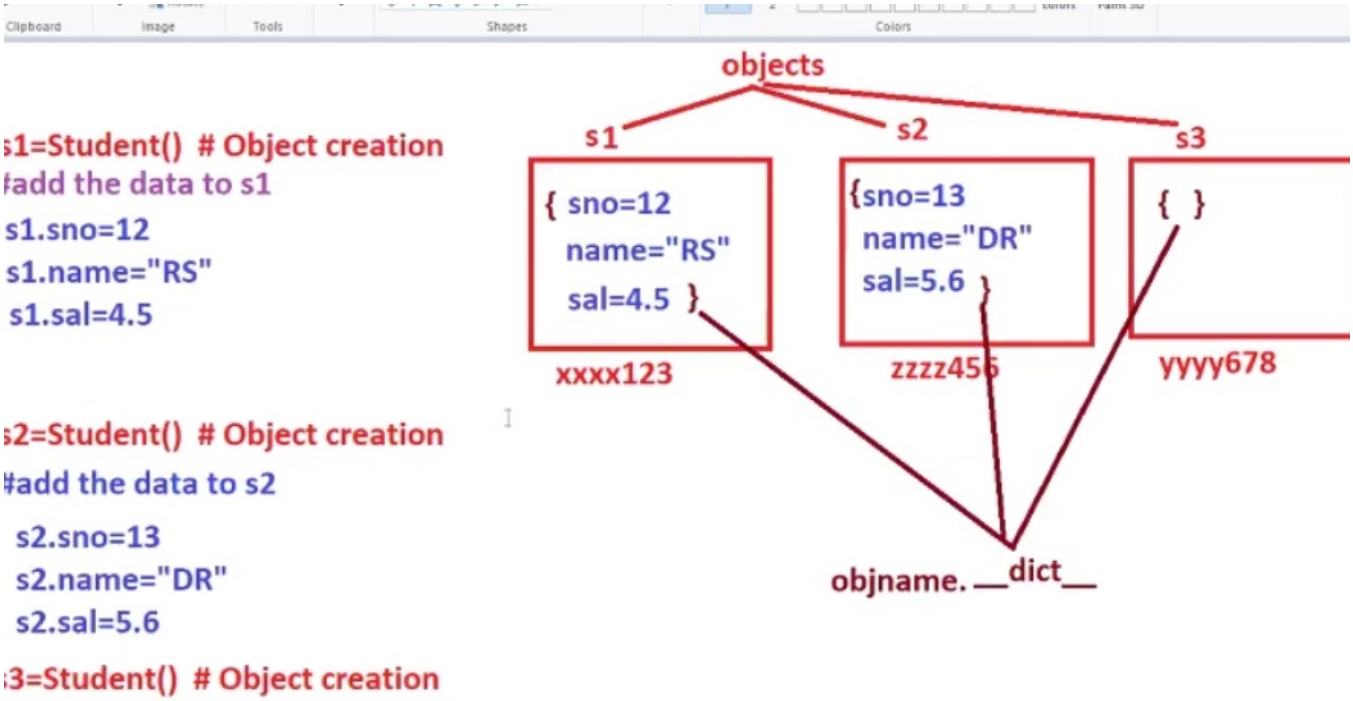
In [9]:
```
from IPython.display import Image
Image(r'C:\Users\shaba\Pictures\Screenshots\Screenshot (187).png')
```

Out[9]:



In [11]:
```
from IPython.display import Image
Image(r'C:\Users\shaba\Pictures\Screenshots\Screenshot (185).png')
```

Out[11]:



3 types of methods in class

i)instance methods

used to performing specific opertations on obj ,also called obj level methods

progmatically instance methods always take 'self' as a first positional formal parameter.

self: implicit obj which is always contains address of current obj.

ii)class level methods

used for performing common operationsfor the obj of currosponding class and specifies class level data members by using @classmethod.

cls: implicit obj and it contains current class name.

cls always used as a first formal parameter in class level method

inside data members must be accesses w.r.t obj name or cls or object name or self.

ii)static methods

used for performing universal operations ,used decorator called @staticmethod and it is never takes 'cls' or 'self' but always takes obj of other classes.

inside data members must be accesses w.r.t obj name or cls or object name or self.

instance_data_member

In [ ]:

```
#program for storing student number,name and marks
#program1
class student:pass
#main
s1=student()
s2=student()
print('contenr of s1 before adding values{} {}'.format(s1.__dict__,len(s1.__dict__)))
```

```
print('content of s2 before adding values{}'.format(s2.__dict__))
s1.sno=100
s1.name='naresh'
s1.marks=89
s2.sno=200
s2.name='suresh'
s2.marks=87
print('contenr of s1 after adding values{} {}'.format(s1.__dict__,len(s1.__dict__)))
print('content of s2 after adding values{}'.format(s2.__dict__))
```

In [ ]:
```
#program for storing student number,name and marks

#program 2
class student:pass
s1=student()
s2=student()
print('content of s1={} and number of values={}'.format(s1.__dict__,len(s1.__dict__)))
print('content of s2={} and number of values={}'.format(s2.__dict__,len(s2.__dict__)))
s1.sno=int(input('enter a student number for content of s1'))
s1.sname='surya'
s1.smarks=34.5
s2.sno=200
s2.sname='mahesh'
s2.smarks=43.9
for k,v in s1.__dict__.items():
    print(k,v)
for k,v in s2.__dict__.items():
    print(k,v)
```

In [ ]:
```
#program 3
class student:pass

#main
#adding instance data
s1=student()
s1.no=1
s1.name='jaya'
s1.marks=89

print(s1.no)
print(s1.name)
print(s1.marks)
 #printig this way is said instance data member32 main purpose is to acces
 #value of class level data mmbr
```

In [ ]:
```
#program 4
class student:
    crs='python' #here crs is class level data member
#main
s1=student()
s1.no=1
s1.name='jaya'
s1.marks=89

print(s1.no)
print(s1.name)
print(s1.marks)
print(student.crs) #printig this way is said instance data member,
                    #main purpose is to acces value of class level data mmbr
```

class_level_data_member

In [ ]:
```
class student:
    crs='python' #here course is called class level data member
s1=student()
s2=student()

#adding instance data to s1 obj through an obj
s1.no=int(input('enter s1 student number ='))
s1.name=input('enter s1 student name =')
s1.marks=float(input('enter s1 a student marks ='))

#adding instance data to s2
s2.no=int(input('enter s2 student number ='))
s2.name=input('enter s2 student name =')
s2.marks=float(input('enter s2 a student marks ='))
```

```
#display first obj data
print('='*50)
print('FIRST OBJECT DATA')
print('='*50)
print(s1.no)
print(s1.name)
print(s1.marks)
print(student.crs)


#display second obj data
print('='*50)
print('SECOND OBJECT DATA')
print('='*50)
print(s2.no)
print(s2.name)
print(s2.marks)
print(student.crs)
```

instance_method

In [ ]:
```
# PROGRAM 1
class student:
    def readstudvalues(self):
        self.no=int(input('enter s1 student number ='))
        self.name=input('enter s1 student name =')
        self.marks=float(input('enter s1 a student marks ='))
    def dispstudvalue(self):
        print(self.no)
        print(self.name)
        print(self.marks)
#main
s1=student()
s1.readstudvalues()
s1.dispstudvalue()
```

In [ ]:
```
# PROGRAM 2
class sum:
    def readvalues(self): #instance method
        self.a=int(input('enter a first values ='))
        self.b=int(input('enter a second value ='))
    def sumop(hydself):  #instance method
        hydself.c=hydself.a+hydself.b
    def dispvalues(self): #instance method
        self.readvalues() #calling instance method from another instance method
        self.sumop()
        print('sum of b = {}+ {} = {}'.format(self.a,self.b,self.c))
#main
s=sum()
s.dispvalues()
```

class_level_method

In [ ]:
```
# program 1
class course:
    city='hyd'
    @classmethod
    def coursedet(a):
        a.no='10'
        a.name='surya'
        course.batch='2 pm'
        print(a.no)
        print(a.name)
        print(course.batch)
        print(course.city)
course.coursedet()
```

In [ ]:
```
# program 2
class student:
    @classmethod
    def getcity(cls):
        cls.city='hyd'
    @classmethod
    def crsdet(k):
        k.intname='Naresh IT'
        k.crs='python'
    def readdet(self):
```

```
        self.no=100
        self.name='alina'
        self.marks=98
    def dispdet(self):
        print(self.no)
        print(self.name)
        print(student.intname)
        print(student.crs)
        print(student.city)
student.getcity()
student.crsdet()
s1=student()
s2=student()
s1.readdet()
s1.dispdet()
s2.readdet()
s2.dispdet()
```

In [ ]:
```
# program 3
class student:
    @classmethod
    def getcity(cls):
        cls.city='hyd'
    @classmethod
    def getcoursedet(cls):
        cls.crs='python'
        student.intname='Narsh iT'
        cls.getcity()# calling class level method w.r.t cls
    def readstudvalues(self):
        self.sno=int(input('enter a student number ='))
        self.name=input('enter a student name =')
        self.marks=float(input('enter a student marks ='))
    def dispstudvalues(self):
        print('student number = {}'.format(self.sno))
        print('studetn name = {}'.format(self.name))
        print('studetn marks ={}'.format(self.marks))
        print('studetn course ={}'.format(student.crs))
        print('student institute ={}'.format(student.intname))
        print('student city ={}'.format(student.city))
#main
s1=student ()
s1.getcoursedet()
s2=student()
print('='*50)
print('ENTER FIRST STUDENT VALUES ')
print('='*50)
s1.readstudvalues()
print('='*50)
print('ENTER SECOND STUDENT VALUES ')
print('='*50)
s2.readstudvalues()
print('='*50)
print('FIRST OBJ OF S1')
print('='*50)
s1.dispstudvalues()
print('='*50)
print('SECOND OBJ OF S1')
print('='*50)
s2.dispstudvalues()
```

static_method

In [ ]:
```
#program 1
class human:
    @staticmethod
    def dispvalues(obj):
        for k,v in obj.__dict__.items():
            print('data =',(k,v))
#main
emp=human()
student=human()
teacher=human()
#adding data
emp.eno=int(input('enter a emp number ='))
emp.ename=input('enter a emp name =')
emp.sal=float(input('enter a emp salary ='))
student.no=int(input('enter a student number ='))
student.name=input('enter a studetn name =')
student.marks=float(input('enter a student marks ='))
teacher.tno=int(input('enter a teacher number ='))
teacher.tname=input('enter a teacher name =')
```

```
teacher.sub=input('enter a teacher subject =')
human.dispvalues(emp)
human.dispvalues(student)
human.dispvalues(teacher)
```

In [ ]:
```
# program 2
class student:
    def readstudvalues(self):
        self.no=int(input('enter a student number='))
        self.name=input('enter a student name =')
class emp:
    def readempvalues(self):
        self.eno=int(input('enter a emp number ='))
        self.ename=input('enter a emp name =')
        self.esal=float(input('enter a emp salary ='))
class teacher:
    def readteachvalues(self):
        self.tno=int(input('enter a teacher number ='))
        self.tname=input('enter a teacher name =')
        self.tsal=float(input('enter a teacher salary ='))
class alina:
    @staticmethod
    def dispobjdata(obj,objinfo):
        print('{} information'.format(objinfo))
        for k,v in obj.__dict__.items():
            print('\t{}\t{}'.format(k,v))
#main
s=student()
e=emp()
t=teacher()
s.readstudvalues()
e.readempvalues()
t.readteachvalues()
a=alina()
a.dispobjdata(s,'student')
a.dispobjdata(e,'employee')
alina().dispobjdata(t,'teacher')
```

CONSTRUCTORS IN PYTHON

The purpose of constructors in python is that 'to initilize the object'.

initilizing the obj is nothing but placing our own data in object without leaving object empty.

definition

A constructor is one of the special method which is automatically /implicitly called by PVM during oject creation and whose purpose is to initilize the object.

syntax:

```
def __init__ (self,list of formal params if any):
 - - - - - - - - - - - - - - - - - - - - - - - - - -
 block of statements
 performs initlization
 - - - - - - - - - - - - - - - - - - - - - - - - - -
```

TYPES OF CONSTRUCTORS

1.Default or parameter less constructor

2.Paraneterized constructor

DEFAULT OR PARAMETER LESS CONSTRUCTOR

A default is one ,which never takes any formal parameters except self.

The purpose of default constructor is that 'to initilize multiple objs of same class with same values'.

syntax:

```
def __init__ (self):
- - - - - - - - - - - - - - - - -  - - -  - - -
block of statements
performs initlization
- - - - - - - - - - - - - - - - -  - - -  - - -
```

In [ ]:
```python
#  example of default constructor

class test:
    def __init__(self):
        self.a=10
        self.b=20
        self.c=self.a+self.b
        print(self.c)
t1=test()
```

PARAMETERIZED CONSTRUCTOR

A parameterized constructor is one, which always takes formal parameters after self.

The purpose of parameterized constructor is that 'to initilize multiple objects with same class with different values.

syntax:

```
def __init__ (self,list of formal params if any):
 - - - - - - - - - - - - - - - - -  - - -  - - -
 block of statements
 performs initlization
 - - - - - - - - - - - - - - - - -  - - -  - - -
```

In [ ]:
```python
# example of parameterized constructor
class test:
    def __init__(self,k,v):
        self.a=k
        self.b=v
        self.c=self.a+self.b
        print(self.c)
t1=test(12,34)
t2=test(10,54)
t3=test(10,7)
```

default parameter constructor

In class of python we can not define both default and parameterized constructor bcoz PVM can rember only latest constructor (due to its interpretation process) ,To full fill the need of both default and parameterized constructors ,we define single constructor with default parameter mechanism.

In [ ]:
```python
class test:
    def __init__ (self,k=1,v=2):
        self.a=k
        self.b=v
        print(self.a,self.b)
#main
t1=test()
t2=test(10,20)
```

In [ ]:
```python
class test:
    def __init__ (self,a=1,b=2):
        self.a=a
        self.b=b
        print('value of a ={}'.format(self.a))
        print('value of b ={}'.format(self.b))
#main
t1=test() #obj creation calls --default cons
t2=test(10,20) # obj creation calls-- parameterized cons
t2=test(100) # obj creation calls-- parameterized cons
t2=test(a=100,b=200) # obj creation calls-- parameterized cons
t2=test(b=300,a=400) # obj creation calls-- parameterized cons
t2=test(200) # obj creation calls-- parameterized cons
```

In [ ]:

```
# program for non-constructor
class employee:
    def getempvalues(self,eno,ename):
        self.eno=eno
        self.ename=ename
e=employee()
print('content of before adding the data',e.__dict__)
e.getempvalues(10,'rossum')
print('content of after adding the data',e.__dict__)
```